

Optimizing RAG Pipelines for Small Language Models:

Evaluating Document Preprocessing, Chunking, and Query Expansion on Structured Administrative Data

Daniel Gelderblom

May 2026

1 Introduction

Since the publication of OpenAI’s GPT-3, Large Language Models (LLMs) have gained immense importance. A major problem with LLMs, however, is their tendency to hallucinate. Especially in contexts like user queries on specific documents, factual correctness is vital.

Before the rise of transformer-based LLMs, language models had received all the knowledge during training. Guu et al. (2020) first used retrieval from a knowledge base together with generation during training, thus not needing to train the model on all knowledge. Karpukhin et al. (2020) improved dense retrieval (retrieval based on similarity scores of text embeddings), making it better than keyword-based retrieval and thus completing the foundation for Retrieval Augmented Generation (RAG). RAG (P. Lewis et al. 2020) combines a (most often dense) retriever with a generational model. The retriever takes documents from a knowledge base relevant to a query and feeds them to the text generator to answer the query.

As context windows have increased to one million tokens, the need for RAG systems for cutting edge proprietary LLMs, while still present, has decreased. However, these LLMs cannot serve every use-case. When data is confidential, privacy is crucial, and only a locally running language model can be considered. In these settings, resource constraints often apply. Thus, only Small Language Models (SLMs) are an option here. SLMs have smaller context windows (262,144 tokens for the very current Qwen3.5-2B). Far more importantly, due to their small size, they suffer much more from lost-in-the-middle problems. Lost-in-the middle describes the phenomenon where transformer-based language models fail drastically when the answer to a question is placed in the middle of a long prompt (Liu et al. 2024). Thus, performance often drops far before reaching the context limit. For SLMs below 10B parameters, this happens as early as 4000 to 8000 input tokens (Leng et al. 2024). This means that only a few short documents can be part of the prompt, increasing the importance of a well-designed RAG pipeline, and especially of the retrieval step. Improving this step, specifically through document preprocessing, chunking, and query expansion, is what this work focuses on.

Preprocessing is the conversion of documents into a form that is easy for the LLM to understand. Chunking is the technique employed to split the documents, which are all

larger than 8000 tokens, into smaller pieces. Query expansion means transforming the query into a form that is easier for the retriever to handle. Thus, the first two parts of the pipeline are only done once for all documents in the offline phase, while query expansion is done for every user input in the online phase (Figure 1).

The task at hand is to create a chat-bot to answer administrative questions from students for the department of Computational Linguistics. The long-term goal is to have this chat-bot run locally on a small GPU. Therefore, only SLMs can be used as generators. Thus, a RAG system with administrative documents as knowledge base is needed. As these documents are in part structured in tables, document preprocessing is a crucial step.

Thus, there are several goals for this thesis. Firstly, a RAG pipeline is built. This pipeline should be modular, so new components can be easily added and existing components can be easily improved and changed. Secondly, the best set-up for preprocessing the structured documents is determined empirically. Thirdly, the benefits of query expansion are investigated. Fourthly, possible hyperparameters are evaluated for the chunking algorithms and the best chunking method is determined for the context at hand.

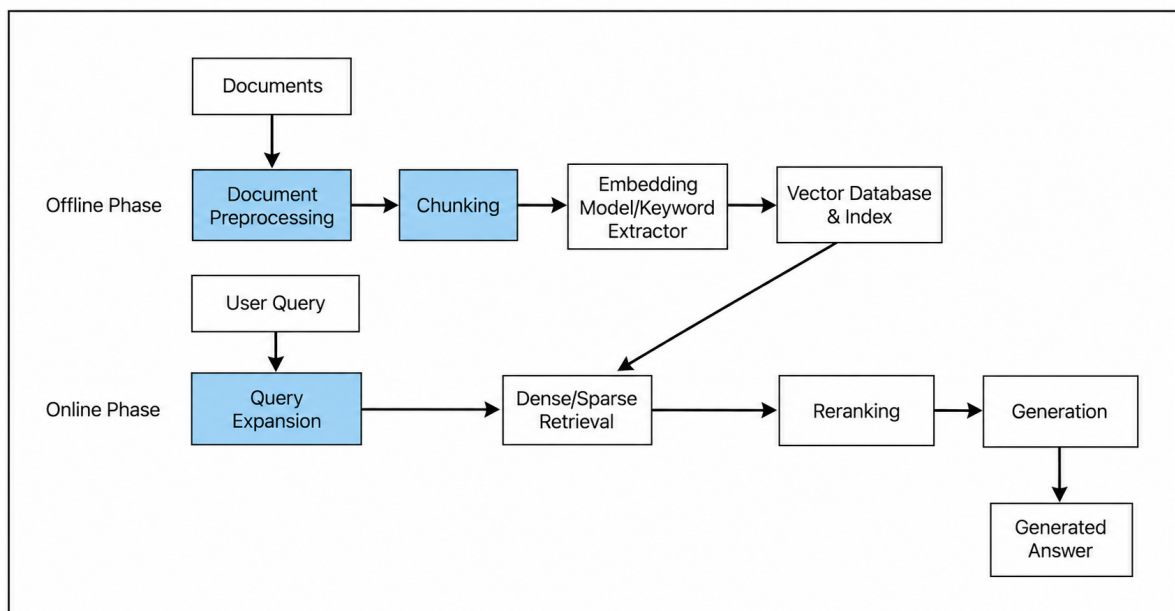


Figure 1: The RAG pipeline

2 Methods

2.1 The documents

The RAG-chatbot is based on three administrative documents from the department of Computational Linguistics at the University of Düsseldorf: Prüfungsordnung (PO), Modulhandbuch (MH) and fachspezifische Prüfungsordnung (fPO). PO is relatively easy to handle, as it contains all the general information about studying and exams and is structured in paragraphs. MH has to be preprocessed, as all the modules for Computational Linguistics are shown in table-like structures with two columns. For fPO, preprocessing is absolutely crucial as there is a table for every semester containing subjects and their credit points, time required, exams and administrative numbers.

2.2 RAG pipeline

2.2.1 Document preprocessing

As two of the documents contain tables, a direct conversion to raw, unformatted text is not sensible. Min et al. (2024) identified four methods to handle tables for RAG. First, the table can be converted to Markdown. Second, it can be converted to simple sentences using automatic templates. Third, traditional (non-generative) language models can be finetuned for this task. Fourth, LLMs can be used to convert the tables to text. In a comparison of the four strategies, LLM-based conversion performed best, followed by Markdown, which still gave good generation results. Thus, for the RAG pipeline, both Markdown and LLM-based conversion are used. For conversion to Markdown, Docling (Deep Search Team 2024) and conversion by an LLM are tested. For conversion to text by an LLM, the technique from Min et al. and a self-written prompt are compared. In the paper, the table is first converted into a nested bracket structure and then converted to text by the LLM. The self-written prompt (see appendix A) takes the documents in Markdown and converts them directly to text.¹

2.2.2 Chunking

With early transformer models and the first RAG systems, chunking was only done in a fixed size of tokens, words or characters (Dai et al. 2019; P. Lewis et al. 2020). For this thesis, Fixed Character Chunking was implemented as a baseline chunker. With Dynamic Token Chunking and Fixed Paragraph Chunking, two other simple deterministic Chunkers were implemented. Dynamic Token Chunking sets a maximum number of tokens and cuts off at the last sentence ending before the limit. Fixed Character Chunking packs a fixed number of paragraphs into one chunk. Fixed Character Chunking and Fixed Paragraph Chunking are used in combination with both preprocessing techniques, while Dynamic Token Chunking is only used with written text, as it needs sentences for splitting and does not work with Markdown tables.

A big problem of chunking in general, and especially fixed-size chunking, is the splitting of contexts (context problem). A word or a sentence in one chunk can reference parts of the previous chunk or even a more distant one. This information gets lost in the chunking process. The first solution to this problem, which was derived from the training of BERT (Devlin et al. 2019), was overlapping chunks .

Table 1: Comparison of Document Preprocessors

Preprocessor	Num Chunks	Context Recall	Faithfulness	Answer Relevancy	Answer Correctness
DoclingMarkdownProcessor	878	0.833	0.614	0.574	0.454
GeminiMarkdownProcessor	261	0.778	0.667	0.699	0.485
GeminiMarkdownProcessor+DirectLLMProcessor	68	0.700	0.811	0.581	0.531
PaperLLMProcessor	924	0.533	0.787	0.613	0.439

2.3 Experiments

2.3.1 Document preprocessing

2.3.2 Query expansion

2.3.3 Grid search for chunking hyperparameters

3 Results

3.1 Preprocessing choices

4 Limitations and Future Research

As this work focuses on preprocessing, chunking and query preprocessing, the optimization of other pipeline components is neglected. Especially the online part of the pipeline has a lot more potential for optimization.

For the embedder, reranker, and generator, other models can be compared. For this work, only a few models were tested superficially, leaving this question open for future work. The retriever can even be combined with a sparse BM25 retriever. In previous work, this has outperformed a pure dense retriever. However, especially (but not exclusively) when switching to hybrid retrieval, the results for chunking and preprocessing might have to be re-evaluated.

For improved retrieval results, the embedding model can be fine-tuned. As the topic is very specific and the number of documents is very limited, this could boost retrieval quality by a great margin. Furthermore, the query preprocessing can be more precise with fine-tuned embeddings. Query2doc [cite!] achieves a performance gain over HyDE by fine-tuning an embedding model on extended queries.

Furthermore, the values for number of retrieved documents, maximum number of documents returned by the reranker and reranking threshold can be improved for any specific chunker in a grid search. For chatbot deployment, the repository can be made more performant using lazy imports, i. e., moving imports into methods. Thus, imports are only triggered when really needed. By using vllm with a locally saved model instead of HuggingFace, there is an even greater potential gain in performance.

References

Dai, Zihang, Zhilin Yang, Yiming Yang, Jaime G Carbonell, Quoc Le, and Ruslan Salakhutdinov (2019). “Transformer-xl: Attentive language models beyond a fixed-length context”. In: *Proceedings of the 57th annual meeting of the association for computational linguistics*, pp. 2978–2988.

¹Parts of this section have been taken from the previous literature review

- Deep Search Team (Aug. 2024). *Docling Technical Report*. Tech. rep. Version 1.0.0. DOI: 10.48550/arXiv.2408.09869. eprint: 2408.09869. URL: <https://arxiv.org/abs/2408.09869>.
- Devlin, Jacob, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova (2019). “Bert: Pre-training of deep bidirectional transformers for language understanding”. In: *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, pp. 4171–4186.
- Guu, Kelvin, Kenton Lee, Zora Tung, Panupong Pasupat, and Mingwei Chang (2020). “Retrieval augmented language model pre-training”. In: *International conference on machine learning*. PMLR, pp. 3929–3938.
- Karpukhin, Vladimir, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih (2020). “Dense passage retrieval for open-domain question answering”. In: *Proceedings of the 2020 conference on empirical methods in natural language processing (EMNLP)*, pp. 6769–6781.
- Leng, Quinn, Jacob Portes, Sam Havens, Matei Zaharia, and Michael Carbin (2024). “Long context rag performance of large language models”. In: *arXiv preprint arXiv:2411.03538*.
- Lewis, Patrick, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. (2020). “Retrieval-augmented generation for knowledge-intensive nlp tasks”. In: *Advances in neural information processing systems* 33, pp. 9459–9474.
- Liu, Nelson F, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang (2024). “Lost in the middle: How language models use long contexts”. In: *Transactions of the association for computational linguistics* 12, pp. 157–173.
- Min, Dehai, Nan Hu, Rihui Jin, Nuo Lin, Jiaoyan Chen, Yongrui Chen, Yu Li, Guilin Qi, Yun Li, Nijun Li, et al. (2024). “Exploring the impact of table-to-text methods on augmenting llm-based question answering with domain hybrid data”. In: *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 6: Industry Track)*, pp. 464–482.

A Prompts

This is prompt.